Multivariate Time Series Prediction Using Deep Learning

Assessment Report

Name: Mohammedh Afrar

Date: 3 November 2026

1. Introduction _____4
1.1 Problem overview 4
1.2 Objectives.....4
1.3 Summary of results.....4
2. Data Insights _____4
2.1 Dataset overview 4
2.2 Target variable distribution 5
2.3 Daily and weekly patterns 5
2.4 Correlation analysis 6
2.5 Key observations.....7
3. Preprocessing _____7
3.1 Data cleaning.....7
3.2 Missing values 8
3.3 Outliers 8
3.4 Train, validation and test split.....8
3.5 Feature scaling 9
3.6 Sequence preparation for deep learning.....9
4. Feature Engineering _____9
4.1 Features created.....9
4.2 Preventing data leakage 10
4.3 Feature relevance analysis 10
4.4 Final feature set 12
5. Model Design _____13
5.1 Baseline models 13
5.2 LSTM architecture (The final model).....13
5.3 GRU architecture.....13
5.4 CNN-LSTM architecture.....13
6. Results _____14
6.1 Evaluation metrics 14
6.2 Validation results 14

6.3 Test results and comparison with baselines.....15
6.4 Predicted versus actual values.....16
6.5 Final model selection.....18
7. Model Optimization _____18
7.1 Techniques applied.....18
7.2 Best hyperparameters.....19
7.3 Performance before and after optimisation.....20
7.4 Discussion.....20
8. Challenges and Solutions _____21
8.1 Misaligned sequence window.....21
8.2 Overfitting.....21
8.3 Random Forest extrapolation failure.....21
9. Conclusion _____21
9.1 Key findings.....21
9.2 Future work.....22

1. Introduction

1.1 Problem overview

Household energy consumption changes from minute to minute as people cook, wash, heat water and use electronic devices. Accurate short term predictions of this consumption support smart home scheduling (for example running a dishwasher when demand is low) home battery and solar management, demand planning by electricity providers and the detection of unusual consumption.

This project predicts the energy used by household appliances in each 10 minute interval. It uses a multivariate time series at every time step the model sees indoor temperature and humidity from nine locations, outdoor weather measurements, time information and the recent history of energy use.

1.2 Objectives

1. Explore the dataset to understand the target variable, temporal patterns and relationships between variables.
2. Clean and prepare the data, including handling missing values, outliers and feature scaling.
3. Engineer time series features (lags, rolling statistics, cyclical time features) and analyse their relevance.
4. Build and train deep learning sequence models (LSTM, GRU and CNN-LSTM) and compare them with baseline models (Linear Regression and Random Forest).
5. Optimise the deep learning models using hyperparameter tuning, regularisation and early stopping.
6. Evaluate all models on a held out test period and select a final model.

1.3 Summary of results

The final model is an LSTM network with one recurrent layer of 64 units. On the unseen test period it achieved a mean absolute error (MAE) of 24.13 Wh, a root mean squared error (RMSE) of 57.58 Wh and R^2 of 0.596. This is a 12.3% lower MAE and 5.7% lower RMSE than the Linear Regression baseline and it was the best of all eight evaluated models on every metric.

2. Data Insights

2.1 Dataset overview

The project uses the Appliances Energy Prediction dataset from the UCI Machine Learning Repository. The data was recorded in a low energy house in Belgium using a wireless sensor network, combined with weather data from a nearby weather station.

Data Set Summary

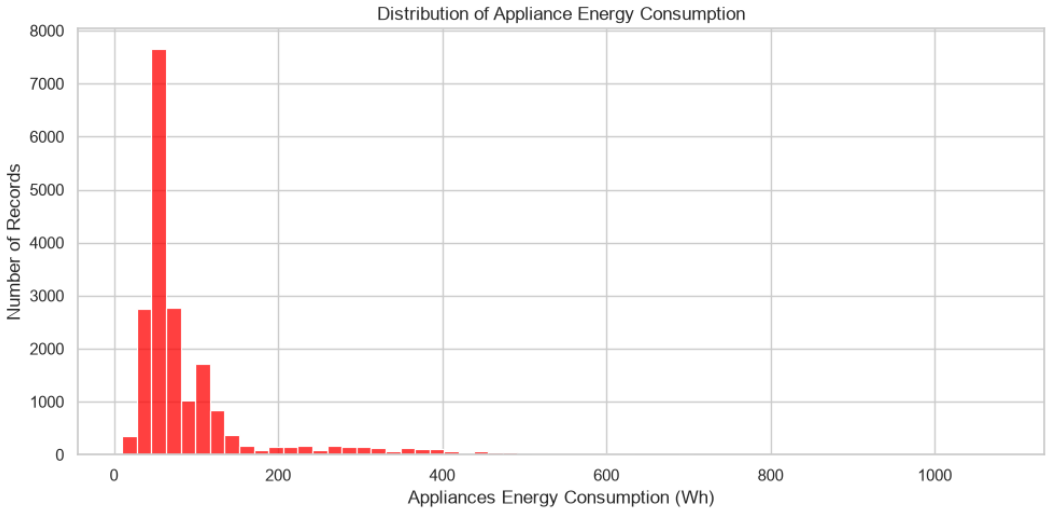
Property	Value
Records	19735
Time period	11 January 2016 17:00 to 27 May 2016 18:00
Sampling interval	10 minutes (All datas)
Columns	29
Missing values / duplicate rows	0 / 0

2.2 Target variable distribution

The target variable is strongly right skewed. Most of the time the house uses little energy but there are short large spikes.

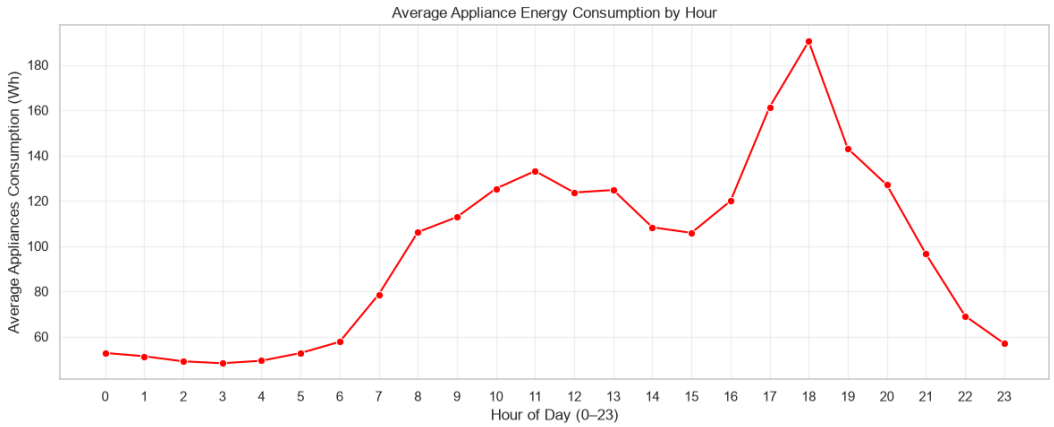
Statistics of the target variable (Appliances)

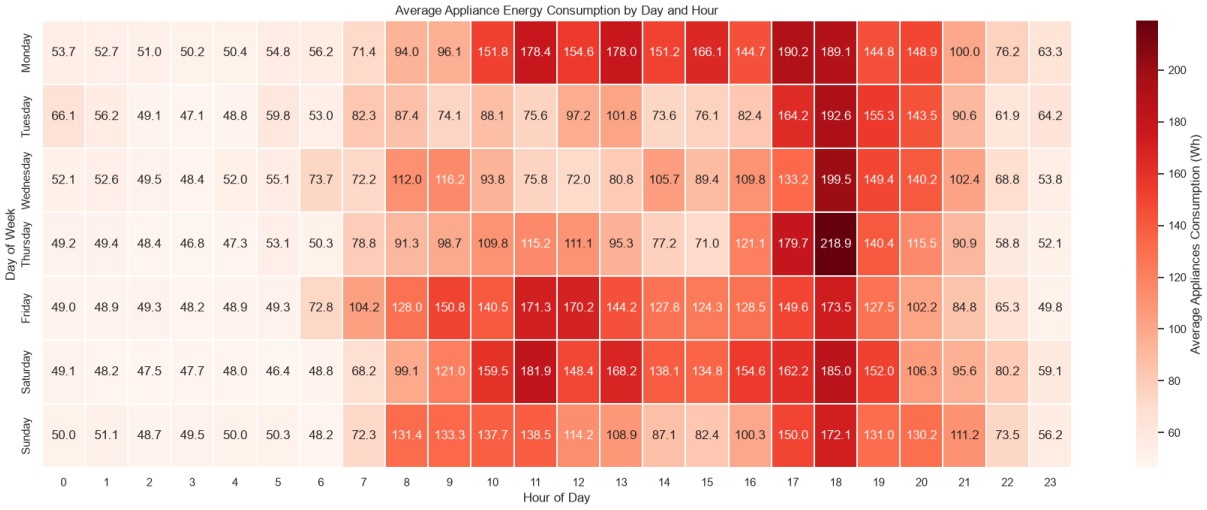
Statistic	Value
Minimum	10 Wh
Median	60 Wh
Mean	97.69 Wh
Standard deviation	102.52 Wh
Maximum	1080 Wh
Skewness	3.39



2.3 Daily and weekly patterns

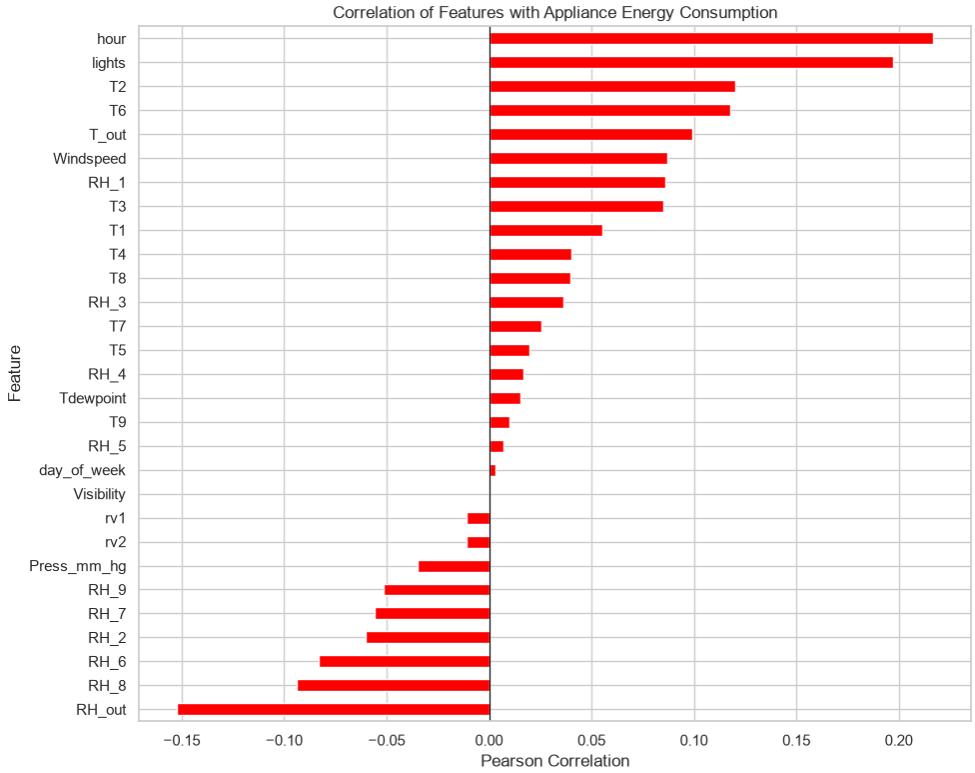
Consumption follows a clear daily routine. It is lowest at night about 48 Wh around 03:00 rises from 07:00 and reaches its highest level in the early evening around 18:00 about 190 Wh on average. The day and hour heatmap shows that the evening peak exists on every day of the week, while daytime consumption is higher on Mondays, Fridays and weekends.





2.4 Correlation analysis

The raw sensor variables have only weak linear correlations with the target. The strongest are hour of the day (0.22), lights (0.20), outdoor humidity (−0.15) and the living room temperature T2 (0.12). This shows that no single sensor explains consumption well and that the recent history of consumption and combinations of features need to be used.



2.5 Key observations

- **Clean data :** no missing values, duplicates or irregular time steps.
- **Skewed target :** a median of 60 Wh but spikes up to 1,080 Wh so the loss function must not be dominated by the spikes.
- **Strong daily cycle :** time of day features are important.
- **Weak individual correlations :** lagged and rolling features based on past consumption are needed.
- **Seasonal change :** the data runs from winter to late spring, so later periods are warmer than earlier ones, which affects how models generalise.

3. Preprocessing

3.1 Data cleaning

- The date column was parsed as a date time and all records were sorted chronologically.
- Duplicate rows and duplicate timestamps were checked (none were found).
- The interval between consecutive records was checked all intervals are exactly 10 minutes so no resampling or gap filling was needed.
- The two random variables rv1 and rv2 were found to be identical. rv2 was removed during feature engineering and rv1 was removed before modelling because it is pure noise.

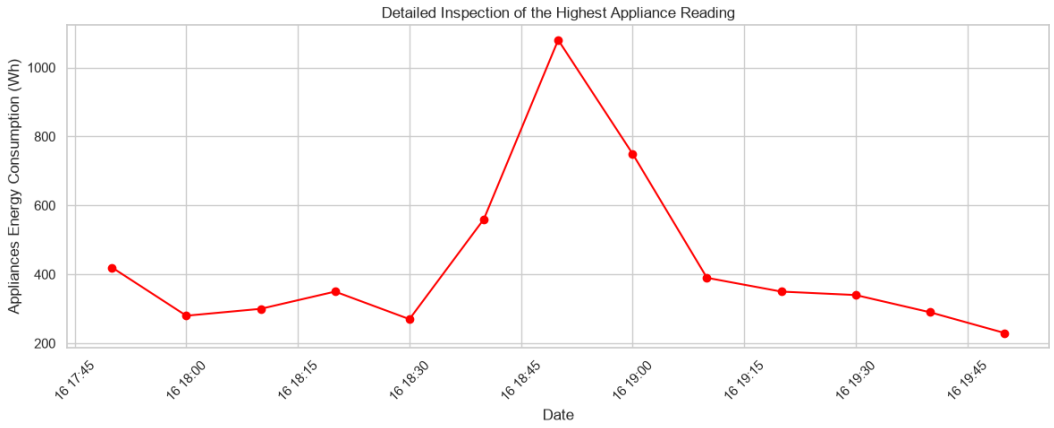
3.2 Missing values

The original dataset contains no missing values in any column so no imputation was required. Creating lag and rolling window features produces missing values in the first rows of the series because there is no earlier history for them. These 18 rows were removed leaving 19717 records. Removing them is better than filling them because any filled value would be an invented history.

3.3 Outliers

Outliers in the target were identified with the Inter Quartile Range (IQR) method. With Q1 = 50 Wh and Q3 = 100 Wh the upper limit is 100 + 1.5 × 50 = 175 Wh. In total 2,138 readings (10.83%) were flagged ranging from 180 to 1080 Wh.

The flagged values were inspected visually. The highest reading is part of a smooth rise and fall in consumption which is consistent with real appliance use rather than a measurement error.



Decision: all outliers were kept.

- They represent genuine high consumption events which the model should learn to predict.
- Removing about 11% of the records would break the regular 10 minute sequence needed for time series models.
- Their influence on training is controlled through the Huber loss function instead of deleting data.

3.4 Train, validation and test split

The data was split chronologically never randomly so that the models are always evaluated on a period that comes after the data they were trained on. A random split would allow the model to train on readings immediately before and after a test reading which gives an unrealistically good score.

Set	Share	Records	Purpose
Training	70%	13801	Fitting model weights
Validation	10%	1972	Early stopping and hyperparameter selection
Test	20%	3944	Final one time evaluation

3.5 Feature scaling

All input features and the target were standardised with scikit learn Standard Scaler which transforms each variable to a mean of 0 and a standard deviation of 1.

- **Fitted on the training set only :** the scalers were fitted on the training data and then applied to the validation and test data so no information from the future leaks into training.
- **Why scaling is needed :** the variables have very different ranges for example pressure around 750 mm Hg and wind speed around 4 m/s. Without scaling large valued features would dominate the gradients and slow down neural network training.
- **Why Standard Scaler instead of MinMax Scaler :** min max scaling depends on the single smallest and largest values so extreme spikes would compress the normal values into a narrow range and test values outside the training range (the warmer May period) would fall outside.
- **Target scaling :** the target was also standardised for stable training and predictions were converted back to Wh before calculating the evaluation metrics.
- **Random Forest :** trained on unscaled features because tree based models are not affected by feature scale.

3.6 Sequence preparation for deep learning

Recurrent models need sequences as input. A sliding window of 6 time steps was created. Each input sample contains the 37 features for time steps t5 to t, and the target is the energy consumption at time t. Because all lag and rolling features only contain past consumption values the window can include the current time step without leaking the target.

Set	Input shape	Target shape
Training	(13796, 6, 37)	(13796,)
Validation	(1972, 6, 37)	(1972,)
Test	(3944, 6, 37)	(3944,)

Validation and test windows at the start of each set use the preceding rows from the previous set only as input context their targets always belong to the set itself.

4. Feature Engineering

4.1 Features created

Feature engineering focused on giving the models information about recent consumption and daily routines because the exploratory analysis showed that raw sensor values alone have weak predictive power.

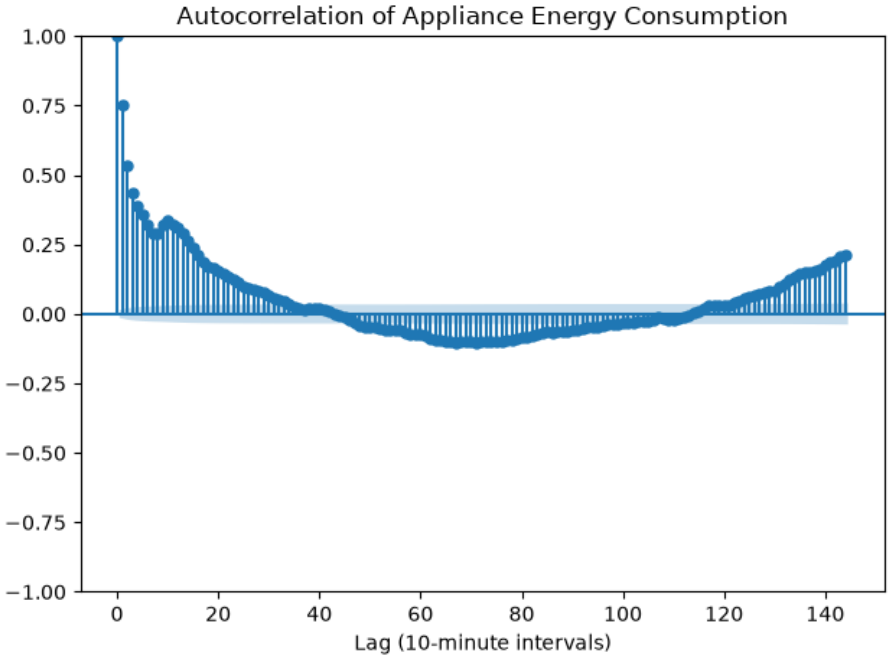
Feature	Definition	Relevance
lag_1, lag_3, lag_6	Appliance consumption 10, 30 and 60 minutes earlier	An appliance running now was probably running 10 minutes ago. lag_1 has the highest correlation with the target (0.76).
rolling_mean_1h	Mean of the previous 6 readings	Recent level of activity in the house.
rolling_mean_3h	Mean of the previous 18 readings	Longer term activity level (for example, an active evening versus a quiet night).
rolling_std_1h	Standard deviation of the previous 6 readings	Volatility of recent consumption high values indicate frequent switching of appliances.
hour_sin, hour_cos	$\sin(2\pi \cdot \text{hour}/24)$ and $\cos(2\pi \cdot \text{hour}/24)$	Captures the daily routine while treating 23:00 and 00:00 as neighbouring hours.
day_of_week_sin, day_of_week_cos	$\sin(2\pi \cdot \text{day}/7)$ and $\cos(2\pi \cdot \text{day}/7)$	Captures the weekly routine Sunday and Monday are neighbours.
is_weekend	1 for Saturday and Sunday, otherwise 0	Occupants are at home for longer at weekends.
temp_humidity_interaction	T1 × RH_1 (kitchen)	Cooking raises kitchen temperature and humidity together. Its correlation with the target (0.09) is higher than that of T1 (0.06) or RH_1 (0.09) alone.

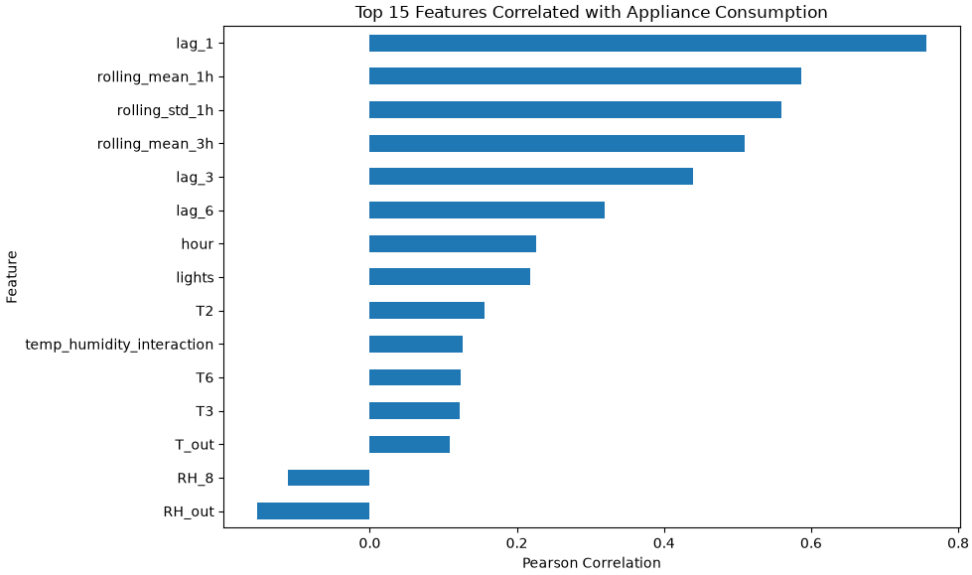
4.2 Preventing data leakage

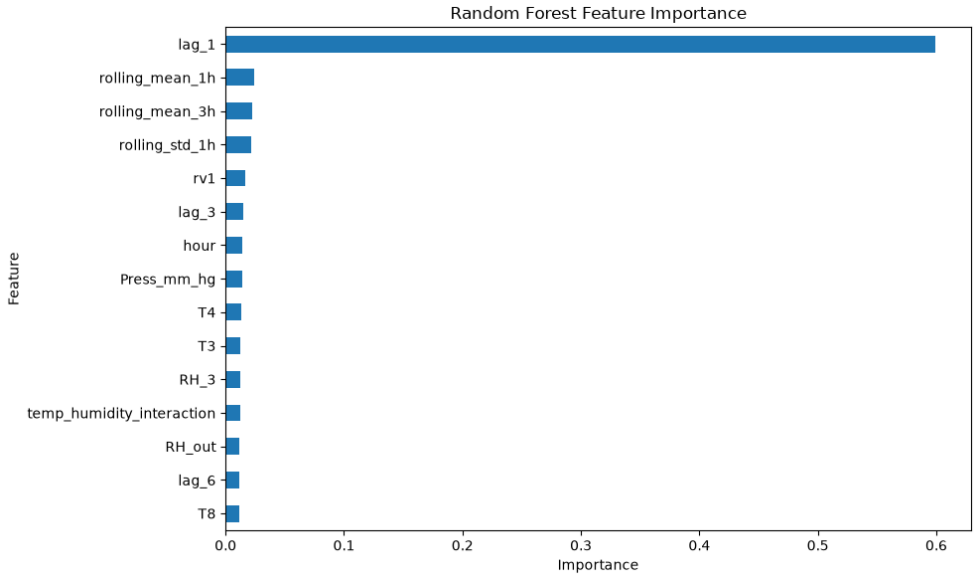
All lag and rolling features were computed after shifting the consumption series by one step. For example the 1 hour rolling mean at 18:50 is the average of the readings from 17:50 to 18:40, and never includes the value at 18:50 that the model predicts.

4.3 Feature relevance analysis

Four methods were used on the training period only : correlation with the target, autocorrelation, Random Forest feature importance and recursive feature elimination (RFE).


Autocorrelation of Appliance Energy Consumption
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
20
40
60
80
100
120
140
Lag (10-minute intervals)


Top 15 Features Correlated with Appliance Consumption
lag_1
rolling_mean_1h
rolling_std_1h
rolling_mean_3h
lag_3
lag_6
hour
lights
T2
temp_humidity_interaction
T6
T3
T_out
RH_8
RH_out
Feature
0.0
0.2
0.4
0.6
0.8
Pearson Correlation



- **Correlation :** the engineered features are the strongest lag_1 (0.76), rolling_mean_1h (0.59), rolling_std_1h (0.56), rolling_mean_3h (0.51) and lag_3 (0.44).
- **Multicollinearity :** two pairs have a correlation of 0.90 or more: T6 and T_out (0.97, since T6 is measured outside) and RH_3 and RH_4 (0.90). Both pairs were kept because they still add some information and the models are regularised.
- **Random Forest importance :** lag_1 had an importance of 0.60 every other feature was below 0.03. The random noise variable rv1 was ranked 5th which shows that impurity based importance can be misleading.
- **RFE with Linear Regression (time-series cross-validation) :** selected only lag_1 (cross-validated MAE 30.2 Wh versus 37.4 Wh with all features).
- **RFE with Random Forest :** selected 15 features, mainly the lag and rolling features, hour, several indoor sensors and pressure.

4.4 Final feature set

The final input contains 37 features lights, 18 indoor temperature and humidity variables, 6 outdoor weather variables, the 6 lag and rolling features, the temperature humidity interaction, is_weekend and the 4 cyclical time features. RFE with Linear Regression selected only lag_1 but this result only reflects linear usefulness. Neural networks can combine features non linearly, so the wider set was kept and its value was confirmed by the final results.

Feature	Reason for removal
rv1, rv2	Random noise added by the dataset authors
month, day_of_month	The training data only covers January to mid April while the test data covers late April to May. These features would force the models to extrapolate to values never seen in training, and day_of_month has no meaningful relationship with consumption.
hour, day_of_week (raw)	Replaced by the sine and cosine encodings.

5. Model Design

5.1 Baseline models

- **Linear Regression :** uses the 37 scaled features of the current time step. It is fast and interpretable and is a strong baseline when lag features are available.
- **Random Forest :** 100 decision trees (random_state = 42) using the unscaled features. It captures non linear relationships and is a standard strong model for tabular data.

Deep learning models are only justified if they outperform these simpler methods so both baselines were evaluated on exactly the same validation and test periods.

5.2 LSTM architecture (The final model)

The Long Short Term Memory network processes the 6 step window in order and maintains a cell state controlled by forget input and output gates. This allows it to learn which recent information is relevant for example a rising consumption trend together with increasing kitchen humidity.

Layer	Configuration	Output shape	Parameters
Input	6 time steps × 37 features	(batch, 6, 37)	0
LSTM	64 hidden units, 1 layer, batch_first	(batch, 6, 64)	26368
Last time step	Output at the final time step	(batch, 64)	0
Dropout	p = 0.2	(batch, 64)	0
Dense	64 → 32, ReLU activation	(batch, 32)	2080
Output	32 → 1, linear activation	(batch, 1)	33
Total			**28481**

5.3 GRU architecture

The Gated Recurrent Unit (GRU) is a simplified recurrent unit with two gates (reset and update) and no separate cell state. It uses the same structure as the LSTM model (64 units → dropout 0.2 → dense 32 with ReLU → linear output) with 21889 trainable parameters. It was included to test whether a lighter recurrent unit gives similar accuracy at lower cost.

5.4 CNN-LSTM architecture

The CNN LSTM model adds a 1 dimensional convolution. Its output sequence (5 steps × 64 channels) is passed to an LSTM with 64 units, followed by dropout 0.2 a dense layer of 32 units with ReLU and a linear output. The model has 40193 trainable parameters.

6. Results

6.1 Evaluation metrics

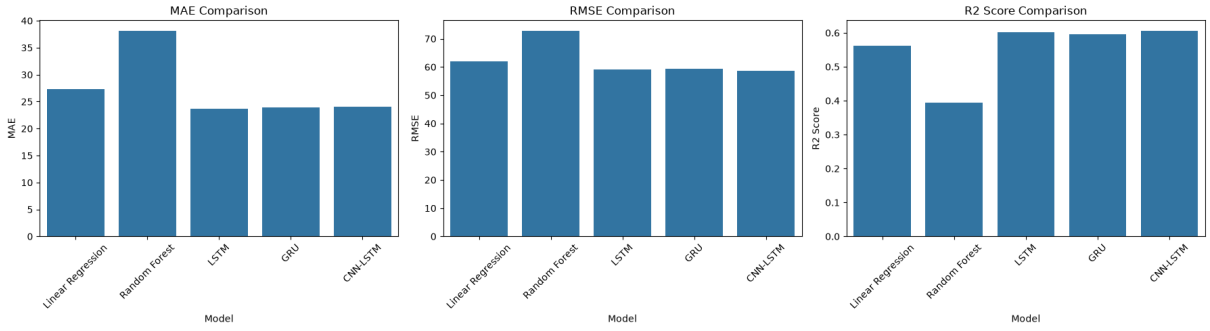
- **MAE (Mean Absolute Error) :** the average size of the error in Wh. It is easy to interpret and is not dominated by large errors.
- **RMSE (Root Mean Squared Error) :** penalises large errors more heavily so it shows how well spikes are predicted.
- **R^2 (coefficient of determination) :** the share of the variance in consumption explained by the model (1 is perfect, 0 is no better than predicting the mean).
- **MAPE (Mean Absolute Percentage Error) :** the average error as a percentage of the actual value. It is defined here because the target is never zero (the minimum in the test set is 20 Wh) but it gives more weight to low consumption periods.

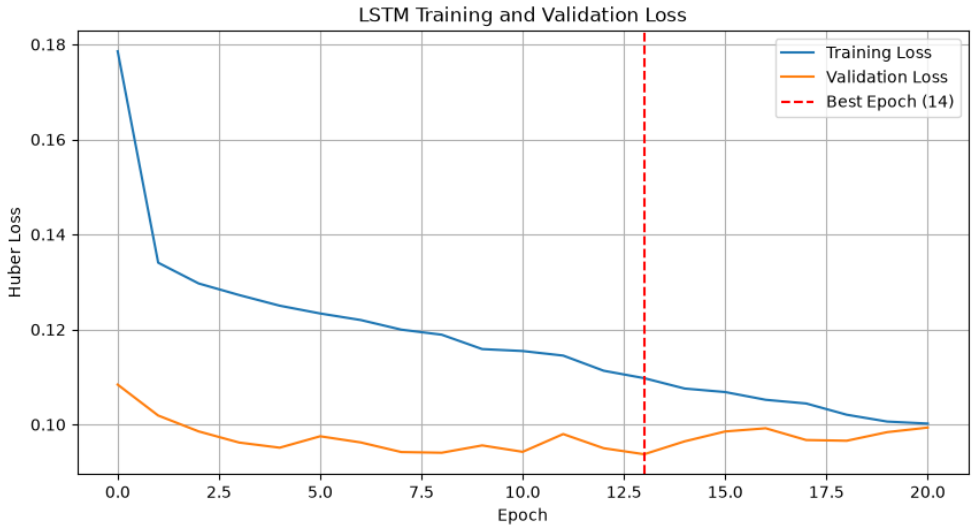
All metrics are calculated in the original unit (Wh) after inverse-transforming the scaled predictions.

6.2 Validation results

Model	MAE (Wh)	RMSE (Wh)	R^2
Linear Regression	27.31	61.89	0.563
Random Forest	38.19	72.91	0.393
LSTM	23.72	59.00	0.603
GRU	23.89	59.46	0.596
CNN-LSTM	24.03	58.69	0.607

All three deep learning models outperformed both baselines on every validation metric. The differences between the three networks are small the LSTM had the lowest MAE and the CNN-LSTM had the lowest RMSE and the highest R^2.





The training loss decreases steadily while the validation loss flattens after a few epochs. The validation loss is lower than the training loss because dropout is active only during training and the validation period has slightly lower average consumption

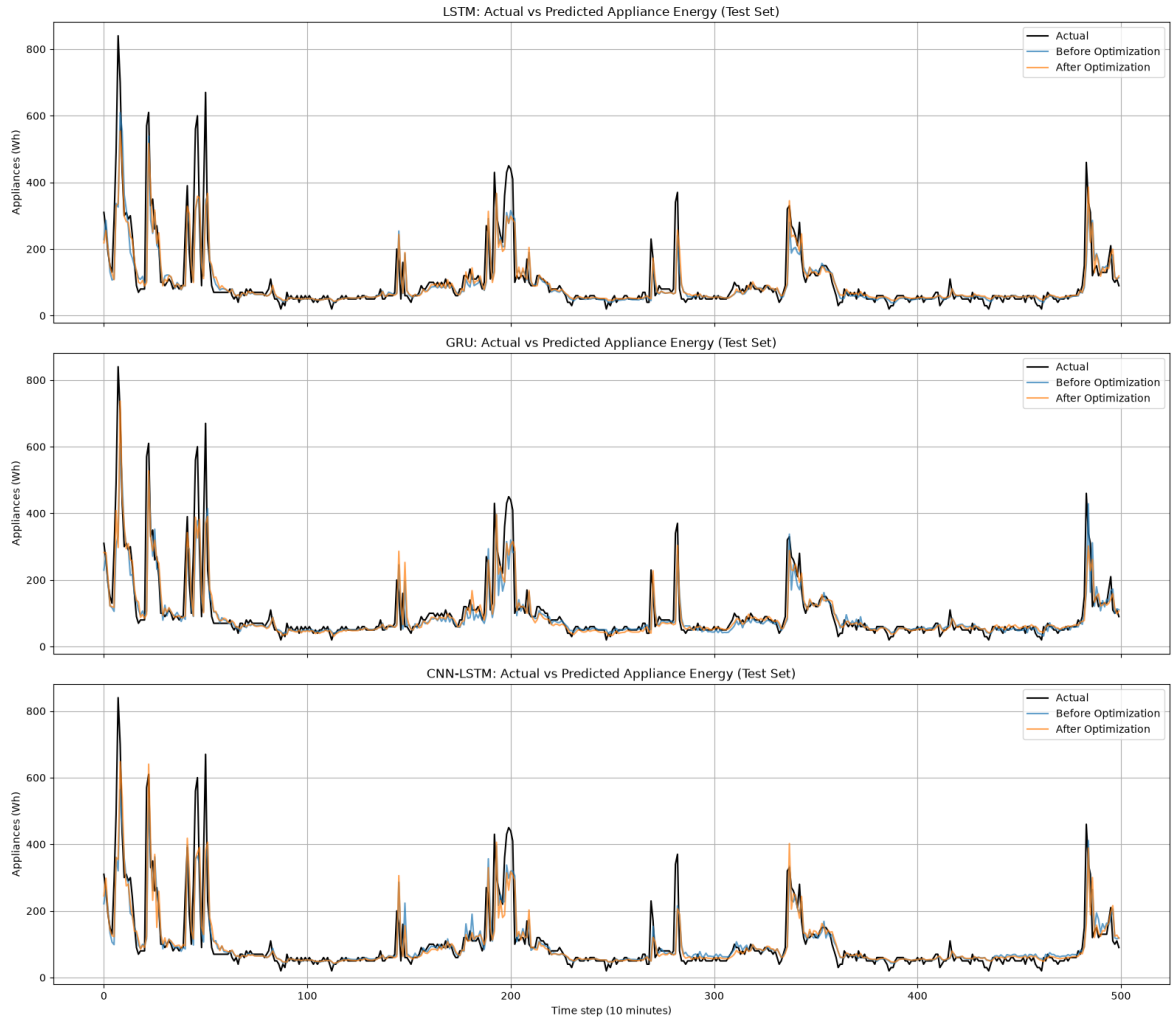
6.3 Test results and comparison with baselines

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
LSTM (selected model)	**24.13**	**57.58**	**20.56**	**0.596**
LSTM (optimised)	24.84	57.98	22.21	0.590
GRU (optimised)	26.20	58.39	25.03	0.584
GRU	25.80	58.57	23.80	0.582
CNN-LSTM (optimised)	24.67	59.59	21.14	0.567
CNN-LSTM	24.87	59.65	20.66	0.566
Linear Regression	27.53	61.07	25.62	0.545
Random Forest	79.17	120.51	87.81	−0.771

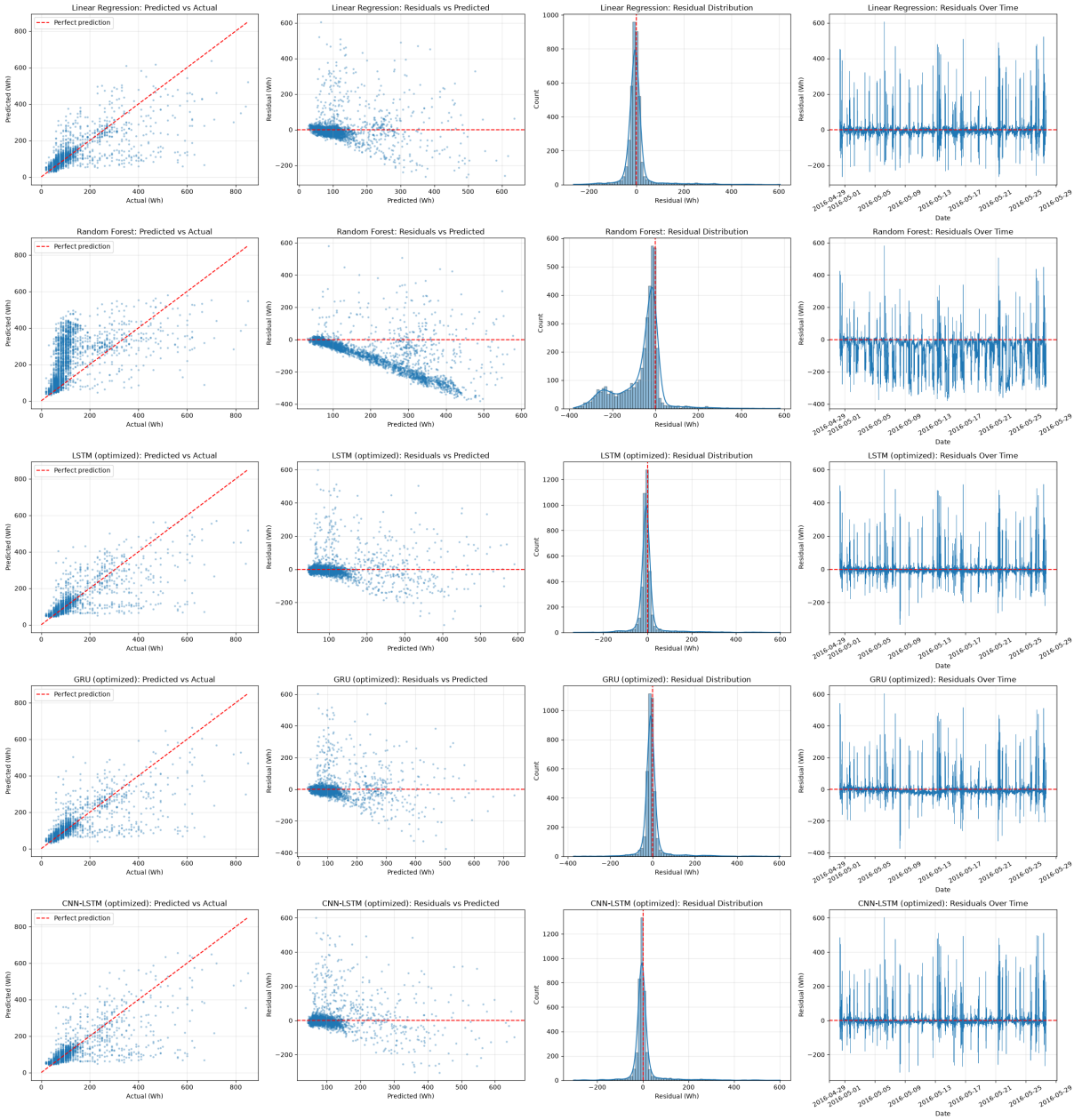
On the test set the LSTM achieved the best result on all four metrics. All three deep learning models out performed both base lines.

Random Forest performed very poorly on the test set (R^2 −0.771), even though it was the second best baseline on validation. The test period is warmer than any part of the training data the outdoor temperature reached 26.1 °C in the test period but only 19.7 °C in the training period.

6.4 Predicted versus actual values



The networks track the normal consumption level closely and follow most changes in consumption. The largest errors occur at sudden spikes the models usually under predict the peak and react one step late because a spike caused by switching on an appliance cannot be anticipated from the previous readings.



Model	Mean residual (Wh)	Median residual (Wh)	Within ±25 Wh	Largest under prediction (Wh)
Linear Regression	−0.14	−5.73	76.9%	605.1
Random Forest	−66.45	−31.90	40.7%	581.1
LSTM (optimised)	0.56	−5.60	80.9%	599.8
GRU (optimised)	−1.46	−7.74	77.8%	603.7
CNN-LSTM (optimised)	3.33	−4.02	82.2%	602.3

- Except for Random Forest all models are nearly unbiased
- The deep learning models predict around 78–82% of the test readings within ±25 Wh, compared with 77% for Linear Regression.

- The largest errors (around 600 Wh) are under predictions of sudden spikes and they occur at the same moments for all models.
- Random Forest shows a systematic negative bias over the whole test period which confirms the extrapolation problem.

6.5 Final model selection

The untuned LSTM was selected as the final model for the following reasons :

1. It achieved the best test MAE, RMSE, MAPE and R^2 of all eight models.
2. It clearly outperforms both baseline models.
3. Hyperparameter tuning did not improve it reliably. On validation the optimised LSTM was essentially tied with the original so the simpler model with one layer was preferred.
4. It is small and fast enough to produce a prediction every 10 minutes on modest hardware.

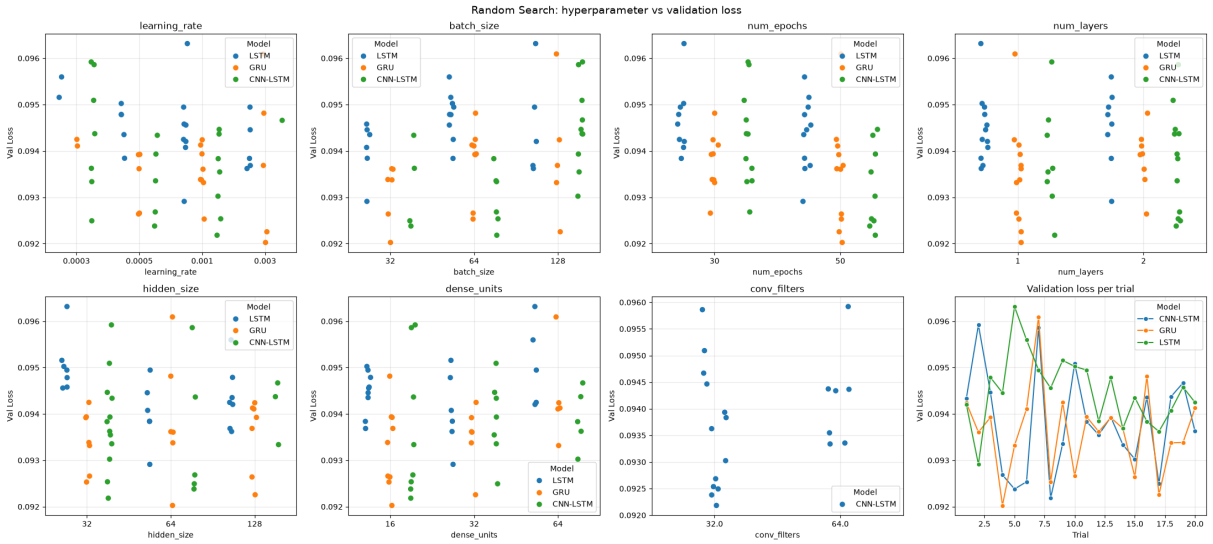
7. Model Optimization

7.1 Techniques applied

Each deep learning model was optimised in three steps using the same training procedure as in model development Huber loss, Adam with weight decay, gradient clipping and the learning rate scheduler so that the results before and after optimisation are comparable.

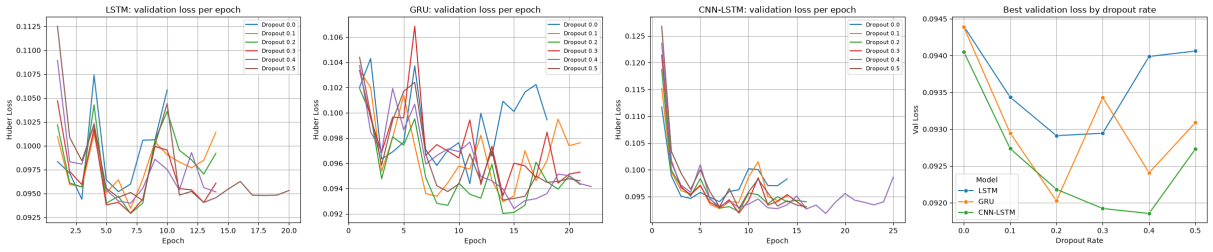
1. **Hyperparameter tuning with random search :** 20 randomly sampled configurations per model were trained and compared on validation loss. Random search was chosen over grid search because the full grid contains 432 combinations for LSTM and GRU and 864 for CNN-LSTM.
2. **Regularisation with dropout :** each model was retrained with its best configuration and dropout rates of 0.0, 0.1, 0.2, 0.3, 0.4 and 0.5. For two layer models dropout was also applied between the recurrent layers.
3. **Early stopping :** training stopped when the validation loss had not improved by more than 0.0001 for 7 epochs and the best weights were restored.

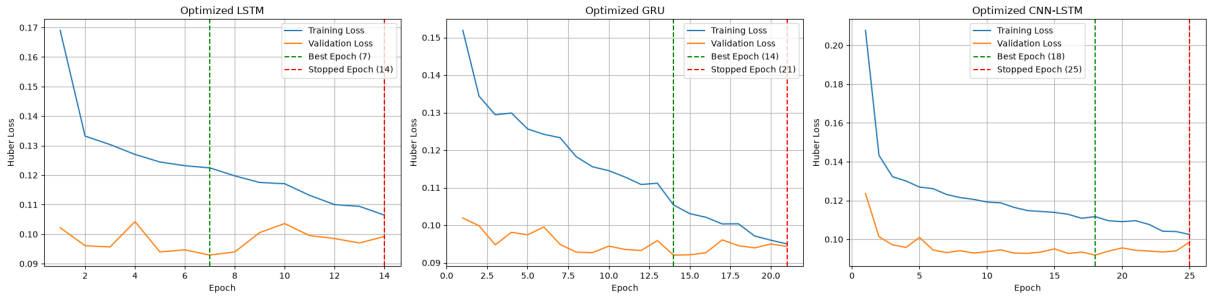
Hyperparameter	Values
Learning rate	0.0003, 0.0005, 0.001, 0.003
Batch size	32, 64, 128
Maximum epochs	30, 50
Recurrent layers	1, 2
Hidden units per recurrent layer	32, 64, 128
Dense layer units	16, 32, 64
CNN filters (CNN-LSTM only)	32, 64



7.2 Best hyperparameters

Hyperparameter	Original (all models)	LSTM	GRU	CNN-LSTM
Recurrent layers	1	2	1	1
Hidden units	64	64	64	32
Dense units	32	32	16	16
Learning rate	0.001	0.001	0.003	0.001
Batch size	64	32	32	64
CNN filters	64 (CNN-LSTM)	–	–	32
Dropout	0.2	0.2	0.2	0.4

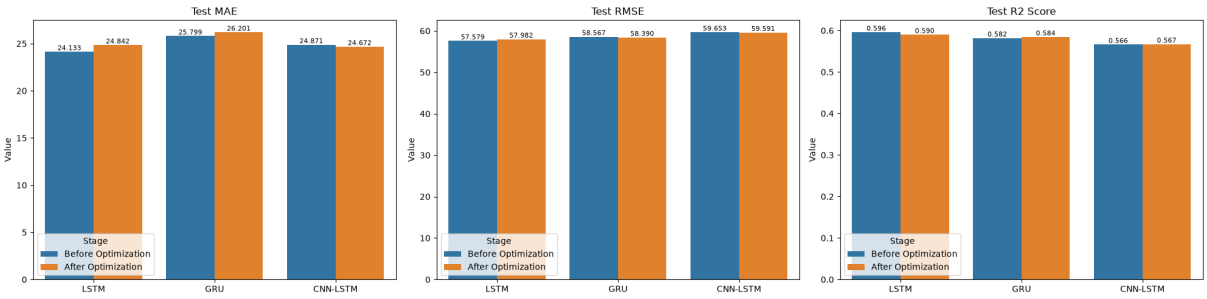




Model	Maximum epochs	Best epoch	Stopped at epoch	Epochs saved
LSTM	50	7	14	36
GRU	50	14	21	29
CNN-LSTM	50	18	25	25

7.3 Performance before and after optimisation

Model	Metric	Before	After	Change
LSTM	MAE (Wh)	24.13	24.84	+2.9% (worse)
LSTM	RMSE (Wh)	57.58	57.98	+0.7% (worse)
LSTM	R^2	0.596	0.590	−1.0% (worse)
GRU	MAE (Wh)	25.80	26.20	+1.6% (worse)
GRU	RMSE (Wh)	58.57	58.39	−0.3% (better)
GRU	R^2	0.582	0.584	+0.4% (better)
CNN-LSTM	MAE (Wh)	24.87	24.67	−0.8% (better)
CNN-LSTM	RMSE (Wh)	59.65	59.59	−0.1% (better)
CNN-LSTM	R^2	0.566	0.567	+0.2% (better)



7.4 Discussion

Optimisation produced only small changes all below 3%. CNN-LSTM improved slightly on all three metrics GRU improved on RMSE and R^2 but not on MAE and the LSTM became slightly worse. The main reasons are :-

- The original configuration was already well regularised (dropout, weight decay, early stopping and learning rate scheduling) leaving little room for improvement.
- The validation set covers only 14 days. Selecting the best of 20 configurations on such a short period partly fits the noise of that period so the selected configuration does not necessarily generalise better.
- Most of the remaining error comes from sudden consumption spikes that cannot be predicted from the available features, which no choice of hyperparameters can fix.

Early stopping was the most effective technique in terms of efficiency saving between 25 and 36 of the 50 maximum epochs for each optimised model.

8. Challenges and Solutions

8.1 Misaligned sequence window

Challenge : in the first version of the model development notebook the deep learning models performed much worse than Linear Regression. Investigation showed that the sliding window used time steps t−6 to t−1 to predict time step t. Because the lag features are already shifted by one step the most recent consumption value (t−1) was not contained anywhere in the input, while the baseline models had access to it. The deep learning models were therefore solving a harder problem.

Solution : the window was changed to time steps t−5 to t, which includes the current row. Together with the feature and training improvements described below this raised the validation R^2 of all three networks to about 0.60.

Model	First version MAE	First version R^2	Final MAE	Final R^2
Linear Regression	27.02	0.557	27.31	0.563
LSTM	36.50	0.317	23.72	0.603
GRU	36.11	0.303	23.89	0.596
CNN-LSTM	35.00	0.295	24.03	0.607

8.2 Overfitting

Challenge : in early runs the validation loss started to increase after only a few epochs while the training loss continued to fall.

Solution : dropout L2 weight decay , gradient clipping, a learning rate scheduler and early stopping with restoration of the best weights were added.

8.3 Random Forest extrapolation failure

Challenge : Random Forest produced a negative R^2 on the test set because the test month is warmer than the training period.

Solution : the cause was identified through residual analysis and temperature ranges and Random Forest was not selected. This also shows why the neural networks which extrapolate more smoothly are better suited to data that changes with the seasons.

9. Conclusion

9.1 Key findings

- Appliance energy consumption is strongly skewed and dominated by short spikes but it follows a clear daily routine with an evening peak around 18:00.
- Engineered features based on past consumption (lags and rolling statistics) are far more informative than the raw sensor readings.
- A chronological split scaling fitted only on training data and leakage free features were essential for a realistic evaluation.

- The LSTM model achieved the best test performance (MAE 24.13 Wh, RMSE 57.58 Wh, R^2 0.596), improving on Linear Regression by 12.3% in MAE and was selected as the final model.
- Hyperparameter optimisation changed performance by less than 3% which indicates that the remaining error is caused mainly by unpredictable consumption spikes rather than by model settings.
- Random Forest failed on the test period because it cannot extrapolate to temperatures outside the training range.

9.2 Future work

- **Multi step forecasting :** predict consumption one hour or one day ahead which is more useful for planning.
- **More robust evaluation :** walk forward cross-validation and training with several random seeds to confirm the ranking of the models.
- **Additional models :** gradient boosting (XGBoost, LightGBM) and attention based models such as the Temporal Fusion Transformer.
- **Prediction intervals :** quantile loss to provide a range of likely consumption instead of a single value.
- **Explainability :** SHAP values to show which features drive individual predictions.
- **More data :** a full year of measurements and data from several houses to cover all seasons and improve generalisation.